



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ingeniería

COMPILADORES
PROYECTO 1. MANUAL DE ANALIZADOR LÉXICO

Emilia Macarena Arriaga Rodríguez

Jesús Alonso Martínez Araujo

Marco Gael Ventura Picazo

Profesora:

M.I. Elba Karen Saenz García

Grupo 1. 2026-1

25 de septiembre de 2025

Índice

1. Introducción	2
2. Análisis del Código	2
2.1. Uso de Librerías	2
2.2. Funciones y Estructura de Datos	3
2.3. Manejo de Archivos: Entrada y Salida	4
2.4. Visualización de Resultados	5
3. Expresiones regulares	6
3.1. Clase 0: Palabras Reservadas	6
3.2. Clase 1: Identificadores	7
3.3. Clases 2 y 3: Constantes Numéricas	7
3.4. Clase 4: Constantes de Cadena	8
3.5. Clases 5, 6, 7 y 8: Símbolos y Operadores	9
3.6. Manejo de Errores	10
4. Ejemplos de ejecución	10
4.1. Clase 0: Palabras Reservadas	10
4.2. Clase 1: Identificadores	12
4.3. Clase 2: Constantes Numéricas	13
4.4. Clase 4: Constantes de Cadena	14
4.5. Clases 5, 6, 7 y 8: Símbolos y Operadores	15
4.5.1. Clase 5: Símbolos Especiales	15
4.5.2. Clase 6: Operadores Aritméticos	16
4.5.3. Clase 7: Operadores Relacionales	17
4.5.4. Clase 8: Operador de Asignación	18

1. Introducción

El propósito fundamental de este proyecto es desarrollar un analizador léxico utilizando la herramienta Flex. Este programa funciona como la primera fase de un compilador. Su tarea principal es leer un archivo de código fuente y segmentarlo en componentes básicos con significado propio, conocidos como tokens.

Por ejemplo, al procesar la línea de código:

```
$miVariable = 10;
```

El analizador la descompone e identifica los siguientes tokens: un identificador (*\$miVariable*), un operador de asignación (=), una constante numérica (10) y un delimitador de fin de sentencia (;).

Para administrar la información de manera eficiente, el programa almacena los identificadores y las constantes (literales) en estructuras de datos separadas: la **Tabla de Símbolos** y las **Tablas de Literales**. Al finalizar el análisis, se genera un reporte detallado con todos los tokens encontrados y el contenido de estas tablas.

2. Análisis del Código

2.1. Uso de Librerías

Para que el programa funcione, necesita herramientas básicas del lenguaje C. Estas se incluyen al principio del archivo.

```
1 %{  
2 #include <stdio.h>  
3 #include <stdlib.h>  
4 #include <string.h>  
5 %}
```

Listing 1: Inclusión de librerías

Cada una de estas librerías aporta un conjunto de funciones importantes:

- **stdio.h:** Se utiliza para todo lo relacionado con la entrada y salida de datos, como abrir, leer y escribir en archivos de texto (fopen, fprintf) y mostrar mensajes en la pantalla (printf).
- **stdlib.h:** Proporciona funciones para la gestión de memoria dinámica. En este código, es fundamental para reservar espacio para las cadenas de texto que se guardan en las tablas mediante la función strdup.
- **string.h:** Contiene funciones para manipular cadenas de texto. Se usa para comparar si dos palabras son iguales (strcmp) o para copiar una palabra de un lugar a otro (strdup).

2.2. Funciones y Estructura de Datos

El código se apoya en varias funciones para organizar y reutilizar la lógica. Las más importantes son las que gestionan las tablas donde se almacenan los componentes léxicos.

```
1 int buscar_o_insertar_identificador(char *ident){
2     for(int i = 0; i < pos_tabla_simbolos; i++){
3         if(strcmp(ident, tabla_simbolos[i].nombre_identificador) == 0){
4             return tabla_simbolos[i].posicion;
5         }
6     }
7
8     int posicion_actual = pos_tabla_simbolos;
9     tabla_simbolos[posicion_actual].posicion = posicion_actual;
10    tabla_simbolos[posicion_actual].nombre_identificador = strdup(ident);
11    tabla_simbolos[posicion_actual].tipo = -1;
12    pos_tabla_simbolos++;
13    return posicion_actual;
14
15 }
```

Listing 2: Función para gestionar identificadores

Para almacenar la información, el programa utiliza una estructura de datos simple, que es un arreglo de structs. Esta implementación es de tamaño fijo (estática), ya que la cantidad máxima de

elementos que puede guardar está predefinida por la constante TAMANIO_TABLA. Por lo tanto, las tablas no pueden crecer de forma dinámica durante la ejecución, entonces tienen un límite de 100 elementos cada una.

Para encontrar un elemento, la función realiza una búsqueda lineal. Esto significa que recorre la tabla elemento por elemento desde el inicio, comparando el identificador buscado con cada uno de los que ya están almacenados. Si lo encuentra, devuelve su posición. Si llega al final sin encontrarlo, lo inserta como un nuevo elemento.

```
1 void imprimir_token(int clase, int pos) {  
2     printf("Token agregado: (%d, %d)\n", clase, pos);  
3     fprintf(apSal, "Token agregado: (%d, %d)\n", clase, pos);  
4     fflush(apSal);  
5 }
```

Listing 3: Función para la impresión de tokens

Esta es una función de apoyo simple. Su única tarea es recibir la información de un token (su clase y su posición) y registrarlo. Lo muestra en la pantalla para que podamos ver el progreso y, al mismo tiempo, lo escribe en el archivo de salida.

2.3. Manejo de Archivos: Entrada y Salida

El programa no trabaja con texto escrito directamente, sino que lee de un archivo y guarda los resultados en otro.

```
1 int main(int argc, char *argv[]){  
2     // ...  
3     if (argc != 2) { // Revisa que se le de un archivo de entrada  
4         // ...  
5     }  
6  
7     yyin = fopen(argv[1], "r");  
8     apSal = fopen("resultados.txt", "w");  
9     // ...
```

```
10  
11 }
```

Listing 4: Apertura de archivos de entrada y salida

El programa está diseñado para ser ejecutado desde la terminal, indicándole qué archivo debe analizar. La línea `yyin = fopen(argv[1], "r");` abre en modo lectura ("r") el archivo de texto que el usuario especifica. Por otro lado, `apSal = fopen("resultados.txt", "w");` crea un nuevo archivo llamado `resultados.txt` en modo escritura ("w"), que es donde se almacenará todo el análisis realizado.

2.4. Visualización de Resultados

Una vez que el analizador ha leído todo el archivo de entrada, el programa muestra un resumen final con las tablas completas.

```
1 // ... dentro de la funcion main(), despues de yylex() ...  
2  
3 printf("\n--- TABLA DE SIMBOLOS ---\n");  
4 fprintf(apSal, "\n--- TABLA DE SIMBOLOS ---\n");  
5 for (int i = 0; i < pos_tabla_simbolos; i++) {  
6 printf("Posicion: %d, Nombre: %s\n",  
7 tabla_simbolos[i].posicion,  
8 tabla_simbolos[i].nombre_identificador);  
9 fprintf(apSal, "Posicion: %d, Nombre: %s\n",  
10 tabla_simbolos[i].posicion,  
11 tabla_simbolos[i].nombre_identificador);  
12 }
```

Listing 5: Impresión final de las tablas de símbolos

Al final de la ejecución, el programa recorre las tablas que fue llenando. Por cada elemento guardado, utiliza dos funciones de impresión:

- **printf:** Muestra la información directamente en la terminal.

- **fprintf:** Escribe la misma información en el archivo resultados.txt.

De esta forma, se obtiene el resultado tanto de forma inmediata en la pantalla como de forma permanente en un archivo.

3. Expresiones regulares

Cada regla está definida por un patrón (generalmente una expresión regular) y una acción a ejecutar cuando se encuentra dicho patrón.

```
1 [ \t\n]+      { /* No hacer nada, simplemente los consume */ }
2
```

Listing 6: Regla para ignorar delimitadores

Esta es la primera regla que aplica el analizador. Identifica cualquier secuencia de uno o más espacios, tabuladores o saltos de línea. La acción asociada está vacía, lo que significa que simplemente los ignora y continúa con el análisis.

3.1. Clase 0: Palabras Reservadas

```
1 "alternative" { imprimir_token(0, 0); }
2 "big"        { imprimir_token(0, 1); }
3 "evaluate"   { imprimir_token(0, 2); }
4 "instead"    { imprimir_token(0, 3); }
5 "large"      { imprimir_token(0, 4); }
6 "loop"       { imprimir_token(0, 5); }
7 "make"       { imprimir_token(0, 6); }
8 "number"     { imprimir_token(0, 7); }
9 "other"      { imprimir_token(0, 8); }
10 "real"      { imprimir_token(0, 9); }
11 "repeat"    { imprimir_token(0, 10); }
12 "select"    { imprimir_token(0, 11); }
13 "small"     { imprimir_token(0, 12); }
14 "step"      { imprimir_token(0, 13); }
```

```

15  "stop"          { imprimir_token(0, 14); }
16  "symbol"       { imprimir_token(0, 15); }
17  "throw"        { imprimir_token(0, 16); }
18

```

Listing 7: Reconocimiento de palabras reservadas

Este grupo de reglas identifica las palabras clave del lenguaje. No usan expresiones regulares complejas, sino cadenas de texto exactas. Cada palabra reservada tiene asignado un token de clase 0 y una posición única. Es crucial que estas reglas aparezcan antes que la regla de identificadores para que Flex las reconozca específicamente.

Ejemplo:

Entrada : loop

Salida : Tokenagregado : (0,5)

3.2. Clase 1: Identificadores

```

1  $[a-zA-Z]+
2

```

Listing 8: Regla para identificadores

Esta regla identifica los nombres de variables o identificadores. Con su respectiva restricción:

- \$: Debe empezar obligatoriamente con el símbolo de dólar.
- $[a-zA-Z]^+$: Seguido de una o más letras, ya sean mayúsculas o minúsculas.

Ejemplo:

Entrada : \$miVariable

Salida : Tokenagregado : (1,0)//La posición 0 es porque es el primer identificador

3.3. Clases 2 y 3: Constantes Numéricas


```

1  /* Clase 2: Enteros */
2  0|([1-9][0-9]*) //Decimal
3  [0o][0-7]+      //Octadecimal
4
5  /* Clase 3: Reales */
6  [0-9].[0-9]+
7

```

Listing 9: Reglas para números enteros y reales

Estas reglas identifican números:

- $0|([1-9][0-9]^*)$: Acepta un 0 solo, o un número que empiece con un dígito del 1 al 9, seguido de cero o más dígitos del 0 al 9. Esto evita números como 0123.
- $[0o][0-7]^+$: Identifica números en base octal, que empiezan con O mayúscula o minúscula, seguida de uno o más dígitos del 0 al 7.
- $[0-9]^*.[0-9]^+$: Busca números con punto decimal. Acepta cero o más dígitos antes del punto, un punto (.) y al menos un dígito después del punto.

Ejemplo:

Entrada : 123

Salida : Tokenagregado : (2,0)

Entrada : 3.1416

Salida : Tokenagregado : (3,0)

3.4. Clase 4: Constantes de Cadena

```

1  "[^"]*"|'[^']*'|\"
2

```

Listing 10: Regla para cadenas de texto

Esta es una regla más compleja para capturar texto. Acepta cadenas que estén entre comillas dobles o simples. La parte `[^"]+` significa cualquier carácter que no sea ni comilla doble ni simple, repetido una o más veces. Esto asegura que captura el contenido correctamente sin salirse a la mitad.

Ejemplo:

Entrada : "HolaMundo"

Salida : Tokenagregado : (4,0)

3.5. Clases 5, 6, 7 y 8: Símbolos y Operadores

```
1  /* Clase 7: Operadores Relacionales */
2  "<="          { imprimir_token(7, 2); }
3  ">="          { imprimir_token(7, 3); }
4  "<"           { imprimir_token(7, 0); }
5
6  /* Clase 5: Simbolos Especiales /
7  [
8
9  (){}.,:;] { / ... */ }
10
11 /* Clase 6: Operadores Aritmeticos /
12 [+*/%]     { / ... */ }
13
```

Listing 11: Reglas para operadores y símbolos

Estas reglas identifican los operadores y símbolos.

- **Operadores Relacionales:** Se definen de forma explícita. Es importante poner los operadores de dos caracteres (como `!=`) antes que los de uno (`!`) para que Flex aplique la regla de la coincidencia más largaz los identifique correctamente.
- **Símbolos y Operadores Aritméticos:** Usan conjuntos de caracteres (`[...]`) para agrupar varios símbolos en una sola regla. Por ejemplo, `[+/*]`

Ejemplo:

Entrada :>=

Salida : Token agregado : (7,3)

Entrada : +

Salida : Token agregado : (6,1)

3.6. Manejo de Errores

```
1      . { fprintf(stderr, "Error Lexico: Simbolo no reconocido '%s'\n",  
2      yytext); }
```

Listing 12: Regla para caracteres no reconocidos

Esta es la última regla y actúa como una red de seguridad. El punto (.) en Flex significa cualquier carácter”. Si el analizador llega a esta regla, es porque el carácter en cuestión no coincidió con ninguna de las reglas anteriores. En ese caso, lo reporta como un error léxico.

4. Ejemplos de ejecución

Se presentan una serie de pruebas para demostrar el funcionamiento del analizador léxico. Cada subsección muestra una captura del archivo de entrada y la salida correspondiente en la terminal, verificando que los tokens son identificados y clasificados correctamente.

4.1. Clase 0: Palabras Reservadas

Se procesa un listado con las 17 palabras reservadas definidas en el lenguaje. Como se observa en la salida, cada una es reconocida y se le asigna un token de la clase 0, con su posición correspondiente.

```
1  $clase_1
2  alternativ
3  big
4  evaluate
5  instead
6  large
7  loop
8  make
9  number
10 other
11 real
12 repeat
13 select
14 small
15 step
16 stop
17 symbol
18 throw
```

Figura 1: Entrada de prueba para palabras reservadas.

```
Token agregado: (0, 0)
Token agregado: (0, 1)
Token agregado: (0, 2)
Token agregado: (0, 3)
Token agregado: (0, 4)
Token agregado: (0, 5)
Token agregado: (0, 6)
Token agregado: (0, 7)
Token agregado: (0, 8)
Token agregado: (0, 9)
Token agregado: (0, 10)
Token agregado: (0, 11)
Token agregado: (0, 12)
Token agregado: (0, 13)
Token agregado: (0, 14)
Token agregado: (0, 15)
Token agregado: (0, 16)
```

Figura 2: Salida en terminal mostrando los tokens de clase 0 agregados.

4.2. Clase 1: Identificadores

Esta prueba verifica que el analizador identifique correctamente las cadenas que cumplen con el formato de identificador (iniciar con \$ seguido de letras). Cada identificador único se agrega a la tabla de símbolos.

```
$clase_dos  
$identificadores  
$Alonso  
$Grupo_Compiladores
```

Figura 3: Entrada de prueba para identificadores.

```
Token agregado: (1, 0)  
Token agregado: (1, 1)  
Token agregado: (1, 2)  
Token agregado: (1, 3)
```

Figura 4: Salida en terminal para los identificadores (clase 1).

4.3. Clase 2: Constantes Numéricas

Se prueba el reconocimiento de constantes numéricas enteras. En la entrada se incluyen números en base decimal y octal. Es importante notar que, debido a la regla, un número como 0432442 se interpreta como un 0 (entero) seguido de un error, ya que no cumple el formato octal (O mayúscula/minúscula) ni el decimal (no puede empezar con cero si tiene más dígitos).

```
26 | o1234567 02136 3432  
27 | 0432442
```

Figura 5: Entrada de prueba para constantes numéricas.

```
Token agregado: (2, 0)
Token agregado: (2, 1)
Token agregado: (2, 2)
Token agregado: (2, 3)
Token agregado: (2, 4)
```

Figura 6: Salida en terminal para números (clase 2).

4.4. Clase 4: Constantes de Cadena

Para esta clase, se valida que el analizador acepte cadenas de texto delimitadas por comillas simples o dobles. Como se especificó en la regla, solo las primeras cuatro cadenas del ejemplo son válidas. Las demás, al no cumplir con el formato (por ejemplo, tener comillas intermedias o no estar cerradas), son marcadas como errores léxicos por el analizador.

```
"Hola mundo"
'a' 'b'
" "
' '
"ejemplos no "validos"
""
""
" 'hola' mundo"
```

Figura 7: Entrada de prueba para constantes de cadena.

```
Token agregado: (4, 0)
Token agregado: (4, 1)
Token agregado: (4, 2)
Token agregado: (4, 3)
Token agregado: (4, 4)
Token agregado: (4, 5)
Error L-@xico: S-imbolo no reconocido 'v'
Error L-@xico: S-imbolo no reconocido 'a'
Error L-@xico: S-imbolo no reconocido 'l'
Error L-@xico: S-imbolo no reconocido 'i'
```

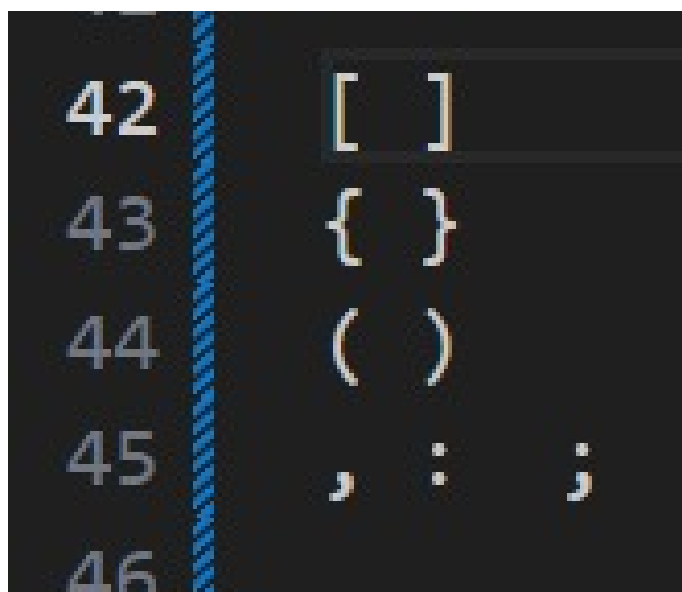
Figura 8: Salida que muestra los tokens de cadena (clase 4) y los errores.

4.5. Clases 5, 6, 7 y 8: Símbolos y Operadores

Finalmente, se verifica el reconocimiento de todos los símbolos y operadores definidos en el lenguaje.

4.5.1. Clase 5: Símbolos Especiales

Se comprueba la correcta identificación de símbolos de agrupación y puntuación como paréntesis, llaves, corchetes, etc.



```
42 [ ]
43 { }
44 ( )
45 , : ;
46
```

Figura 9: Entrada para símbolos especiales.


```
Token agregado: (5, 0)
Token agregado: (5, 1)
Token agregado: (5, 4)
Token agregado: (5, 5)
Token agregado: (5, 2)
Token agregado: (5, 3)
Token agregado: (5, 6)
Token agregado: (5, 7)
Token agregado: (5, 8)
```

Figura 10: Salida para tokens de clase 5.

4.5.2. Clase 6: Operadores Aritméticos

Se identifican los operadores aritméticos básicos.

```
$operadores_aritmeticos
+ - * / % \ ^
```

Figura 11: Entrada para operadores aritméticos.

```
Token agregado: (6, 1)
Token agregado: (6, 5)
Token agregado: (6, 3)
Token agregado: (6, 2)
Token agregado: (6, 4)
Token agregado: (6, 7)
Token agregado: (6, 6)
```

Figura 12: Salida para tokens de clase 6.

4.5.3. Clase 7: Operadores Relacionales

Se reconocen los operadores de comparación, incluyendo los de uno y dos caracteres.

```
50 $operadores_relacionales
51 < > <= >= == !=
```

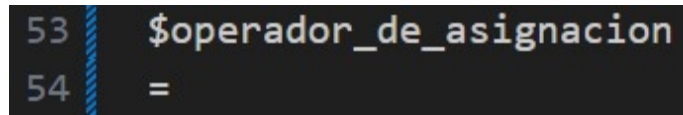
Figura 13: Entrada para operadores relacionales.

```
Token agregado: (7, 0)
Token agregado: (7, 1)
Token agregado: (7, 2)
Token agregado: (7, 3)
Token agregado: (7, 4)
Token agregado: (7, 5)
```

Figura 14: Salida para tokens de clase 7.

4.5.4. Clase 8: Operador de Asignación

Se prueba el símbolo de asignación.



```
53 $operador_de_asignacion  
54 =
```

Figura 15: Entrada para el operador de asignación.



```
Token agregado: (8, 0)
```

Figura 16: Salida para el token de clase 8.